

Mini-projet 14 — Ta première socket TCP

⚠ **Règle d'or, à lire avant tout code réseau.** Tu ne scannes **QUE ce qui t'appartient** : ta propre machine (`127.0.0.1`), ou ton lab que tu contrôles. Scanner une machine tierce sans autorisation est **illégal** (en France, art. 323-1 du Code pénal). Dans tout ce module, la cible est **ta propre machine** — c'est le terrain d'entraînement parfait et sans risque.

Pourquoi ce mini-projet ?
C'est **LE** moment : ton code va vraiment parler au réseau. Une **socket**, c'est une prise de communication — tu t'y branches pour tester si un port répond. Un port qui accepte la connexion = **ouvert** (un service écoute) ; sinon = **fermé**. C'est la brique de base d'un scanner. Et tu vas la tester sur ta propre machine, en ouvrant toi-même un port.

1. C'est quoi une socket, concrètement ?

Une socket TCP, c'est comme **passer un coup de fil** à un couple (IP, port) :

- si quelqu'un **décroche** → le port est **ouvert** (un service écoute derrière)
- si **ça sonne dans le vide / occupé** → le port est **fermé** ou filtré

2. Le code minimal

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
# AF_INET = IPv4 | SOCK_STREAM = TCP (le duo standard)
s.settimeout(1) # n'attends pas plus d'1 seconde par port
code = s.connect_ex(("127.0.0.1", 80)) # tente la connexion
s.close() # on raccroche

if code == 0:
    print("ouvert")
else:
    print("fermé")
```

3. **connect_ex** plutôt que **connect** important

	<code>connect((ip, port))</code>	<code>connect_ex((ip, port))</code>
Port fermé	lève une exception (crash si pas géré)	renvoie un code d'erreur (un nombre)
Port ouvert	ne lève rien	renvoie <code>0</code>

Pour un scanner, `connect_ex` est bien plus pratique : il renvoie **0 si ouvert**, un code non-nul sinon — pas besoin de `try/except` autour de chaque port. Le `_ex` veut dire « version qui renvoie un code au lieu de lever ». (Tu *pourrais* utiliser `connect` + `try/except` — c'est le MP05 — mais `connect_ex` est plus direct ici.)

4. Le timeout : indispensable

Sans `settimeout`, un port filtré peut bloquer ton scan pendant >1 minute (le système attend une réponse qui ne vient jamais). Mets **toujours** un timeout court (1 à 2 s) : `s.settimeout(1)`. C'est ce qui rend un scan rapide.

5. Préparer une cible sur TA machine (2 terminaux)

Pour tester, on ouvre un port bien réel sur `127.0.0.1`. Python le fait en une commande.

Terminal 1 (laisse-le tourner) — ouvre un serveur web local sur le port 8080 :

```
python -m http.server 8080
```

Tant que ce terminal tourne, le **port 8080 est ouvert** sur ta machine. Le port **9999**, lui, ne sera pas ouvert (rien n'écoute dessus) → cible parfaite pour tester le cas « fermé ».

Terminal 2 — c'est là que tu lances ton scanner.

Quand tu as fini, ferme le Terminal 1 avec **Ctrl+C** pour libérer le port.

🎯 Ta mission

- Objectif :** une fonction qui teste si un port est ouvert sur une cible.
- Écris `port_ouvert(ip, port, timeout=1)` qui :
 - crée une socket TCP ;
 - règle le timeout ;
 - tente `connect_ex((ip, port))` ;
 - ferme la socket ;
 - renvoie `True`** si le code vaut `0` (ouvert), sinon `False` .
 - Teste sur ta machine avec le serveur du terminal 1 lancé.

Tests à faire passer (avec `python -m http.server 8080` qui tourne) :

```
print(port_ouvert("127.0.0.1", 8080)) # -> True  (ton serveur écoute)
print(port_ouvert("127.0.0.1", 9999)) # -> False (rien n'écoute)
```

Bonus 1 : réutilise ton générateur `plage_ports` (MP11) pour scanner une plage et n'afficher que les ports ouverts :

```
for p in plage_ports(8078, 8082):
    if port_ouvert("127.0.0.1", p):
        print(f"port {p} ouvert")
# -> port 8080 ouvert
```

Bonus 2 : enveloppe ta socket dans un context manager `with` — une `socket` supporte déjà le protocole `with` (elle a `__enter__` / `__exit__`), donc `with socket.socket(...) as s:` la ferme toute seule (ton MP13 en vrai !).

- ✓ **Critères de réussite**
- `port_ouvert("127.0.0.1", 8080)` renvoie `True` quand le serveur tourne.
 - `port_ouvert("127.0.0.1", 9999)` renvoie `False` .
 - Tu utilises `settimeout` (le scan ne traîne pas).
 - La socket est fermée à chaque appel (via `close()` ou `with`).

Si tu bloques — indices progressifs

Étape	Indice
Créer la socket	<code>s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)</code> (n'oublie pas <code>import socket</code>).
Le couple (ip, port)	<code>connect_ex</code> prend un seul argument : un tuple <code>(ip, port)</code> → double parenthèse : <code>s.connect_ex((ip, port))</code> .
Interpréter	<code>return code == 0</code> — ça renvoie directement <code>True</code> / <code>False</code> , pas besoin de <code>if/else</code> .
Bonus with	<code>with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:</code> — plus besoin de <code>close()</code> , il est automatique.

Où ranger : `mini-projets/14-socket-tcp/scanner.py` .
Quand c'est prêt : `git add .` → `commit` → `push` , puis « c'est poussé » (dis-moi aussi ce que tes 2 tests affichent). On passe ensuite aux timeouts fins puis à la concurrence.